



MPE2 协同覆盖选拔赛赛题

RoboCup 长沙邀请赛组织委员会

(BigHeroX · ROBOT 工作坊)

2026 年 9 月

摘 要：本赛题以多智能体粒子环境为载体，考察参赛者的机器人协同策略设计、强化学习实验与工程交付能力。参赛者需要控制多台机器人在固定时长内持续覆盖若干移动目标区域，在提高覆盖时间占比的同时降低机器人之间的碰撞。本文给出任务的完整定义、观测与动作协议、参赛流程要求、提交材料清单以及分层评分公式。赛题配置了随机、规则、学习三档基线作为成绩锚点，并配套公开评测套件、提交预检工具与示例提交。

关键词：多智能体强化学习；协同覆盖；动态目标；基准相对评分；工程交付

目 录

一、 比赛背景与目的	3
(一) 设计原则	3
二、 任务定义	4
(一) 任务概述	4
(二) 一步的执行顺序	4
(三) 系统边界与信息权限	5
三、 观测、动作与策略接口	5
(一) 观测协议	5
(二) 动作协议	6
(三) 策略生命周期	6
(四) 训练接口	7
四、 参赛要求	7
(一) 参赛流程	7
(二) 环境与运行要求	8
五、 提交材料	9
六、 评分标准	10
(一) 指标定义	10
(二) 团队奖励与回合回报	10
(三) 分层评分公式	10
(四) 难度分组	11
(五) 基线与成绩锚点	11
(六) 成绩类别与最终评定	12
(七) 异常处理	12
七、 附则	12
(一) 版本冻结	12
(二) 扩展规则	13
(三) 成果归档	13

一、比赛背景与目的

多智能体强化学习 (Multi-Agent Reinforcement Learning, MARL) 是当前机器人协同控制研究的重要方向 [1]。为了给本科低年级学生提供一个可上手、可度量、可追溯的智能体协同实践场景, 本组织委员会 (BigHeroX 与 ROBOT 工作坊) 设计了 MPE2 协同覆盖选拔赛。赛题以多智能体粒子环境 (Multi-Particle Environment) 为基础 [1], 面向有编程基础、能够使用人工智能辅助开发的大二学生, 用于观察其机器人协同策略、强化学习实验与工程交付能力。

赛题任务属于**多机器人多目标持续覆盖**方向: N 台机器人在有限观测条件下协同追踪并持续覆盖 M 个移动目标区域。这一任务天然包含三类具有区分度的能力要素:**局部感知下的决策** (每台机器人只能依据自身观测行动)、**多机分工与协作** (机器人之间既需避免重复覆盖, 又需避免碰撞) 以及**完整的实验与交付流程** (训练、保存、加载、评测、报告)。

需要说明的是, 粒子环境的成绩只能反映相关能力的一部分, 不能直接等同于真实机器人比赛能力。赛题的价值在于: 以完全公开、可复现的统一规则, 把“协同策略质量”与“工程交付质量”同时纳入可度量的评价体系。

(一) 设计原则

1. **统一规则与接口:** 任务规则、输入输出协议与评测程序统一提供; 参赛者自由组织策略、网络、训练器和实验。
2. **局部执行、集中训练:** 正式评测严格按单机器人局部观测执行; 训练阶段允许使用全局状态的集中价值估计, 符合“集中训练、分散执行”的主流范式 [1]。
3. **基准相对分数:** 最终成绩相对赛前冻结的随机与学习锚点归一化, 避免绝对分数随任务难度波动。
4. **不改动既有代码:** 参赛者通过统一接口完成训练、保存、加载和评分, 全程不改动任务环境、接口定义与评测程序。

二、任务定义

（一）任务概述

N 个机器人依据各自的局部观测，在固定时长 T 内持续覆盖 M 个移动目标区域，提高覆盖时间占比，同时降低机器人之间的碰撞。任务的核心规则见表 1。

表 1: 任务核心规则（冻结值见配置文件 `configs/task-v1.yaml`）

对象或规则	定案
空间	二维正方形场地，半宽 1.0
机器人	具有半径、速度、驱动力、阻尼和最大速度的粒子； 机器人之间有碰撞响应
目标	具有覆盖半径的非碰撞标记；分段转向、有界速度、 边界反射
覆盖判定	机器人中心进入目标覆盖半径；每步重新计算，离 开后不继续得分
容量	一台机器人每步最多承担一个目标；同一目标每步 最多计一次覆盖
感知	距离范围内目标位置与队友状态可见；不模拟遮挡
执行	每台机器人只能依据自身当前观测及历史记忆输出 动作
通信	首届不设置机器人之间的显式消息通道
回合	固定 T 步（公开套件为 10 步）；全覆盖后继续运行， 考查持续保持能力
规模	同一回合内 N 、 M 固定；不同回合可使用不同规模

这里的“覆盖”统计目标区域被机器人服务的数量；整块场地的扫地面积不进入任何指标。为避免相邻目标使一台机器人重复得分，计分程序按当前几何关系建立机器人—目标二分图，在所有满足覆盖条件的边上求最大匹配，匹配数量即当步覆盖数量；匹配结果只用于计分，不作为分工指令提供给策略。

（二）一步的执行顺序

1. 从状态 s_t 构造所有机器人的观测；

2. 收集所有机器人的动作，收集完成前不推进环境；
3. 推进一次机器人物理与一次目标运动，执行边界处理，得到 s_{t+1} ；
4. 在 s_{t+1} 上计算覆盖与碰撞，生成奖励与下一步观测；
5. 达到 T 步时按时间限制截断，初始状态不单独计奖。

机器人中心被限制在留出自身半径的场内区域，撞击边界时清零朝外法向速度、保留切向速度；目标越界时反射位置与对应速度分量。

（三）系统边界与信息权限

系统划分为四个边界：**任务环境**（组织方持有真实物理状态）、**参赛训练**（可访问全部训练观测、奖励及全局状态）、**正式执行策略**（仅本机器人观测、公开规格与局部历史）与**评测归档**（组织方持有私有测试实例与提交身份）。训练可以使用集中价值网络利用全局状态估计团队长期收益，但正式执行动作的策略仍只使用局部信息；覆盖、碰撞、奖励与分数均由组织方计算，参赛者自定义“分数”不进入核验流程。

三、观测、动作与策略接口

（一）观测协议

协议版本为 `coverage-policy/1.0`。观测容量 $A = 8$ 台机器人、 $B = 8$ 个目标已冻结。所有机器人使用相同形状的观测结构，实际数量通过有效标记表达。单机器人观测字段见表 2。

观测归一化基准固定：距离与位置除以场地半宽 L ，速度除以机器人最大速度 V ，不依赖本局数据缩放。不存在或不可见的实体数据行清零，由两个标记区分这两种状态。目标离开视野后评测程序不再提供其位置，历史记忆由参赛者策略自行维护；目标速度不在执行观测中提供，允许策略从历史位置估计运动趋势，当前目标速度仅提供给训练用全局状态。

任务库提供观测展平辅助函数 `flatten_observation`（展平后 104 维，顺序与版本固定），初学者可配合普通全连接网络使用；有经验的参赛者也可以自行把实体表组织成序列或图结构。

表 2: 单机器人观测字段（类型定义见 `coverage_bench/protocol.py`）

字段	类型与形状	含义
<code>self_state</code>	<code>float32 [5]</code>	自身位置 x, y ，速度 v_x, v_y ，半径
<code>peers</code>	<code>float32 [A,5]</code>	各队友相对位置 2、相对速度 2、半径 1
<code>peer_exists /</code> <code>peer_visible</code>	<code>bool [A]</code>	队友存在与可见标记
<code>targets</code>	<code>float32 [B,3]</code>	各目标相对位置 2、覆盖半径 1
<code>target_exists /</code> <code>target_visible</code>	<code>bool [B]</code>	目标存在与可见标记
<code>agent_index</code>	<code>int</code>	本机器人在回合内的稳定编号
<code>step_index</code>	<code>int</code>	已推进步数，从 0 开始
<code>time_remaining</code>	<code>float32 标量</code>	剩余步数占 T 的比例

（二）动作协议

正式动作统一为 `float32[2]`： x 、 y 两个方向的归一化驱动命令，每个分量取值于 $[-1, 1]$ ，物理适配器把命令转换为已声明的驱动力。惯性与阻尼继续生效，命令为零时机器人按当前速度继续运动；物理引擎另行限制最大速度。错误形状、非有限数值与超出范围均按协议错误处理，评测程序不静默裁剪。参赛者可在内部使用离散动作（如五个方向），再由本人入口映射为这两个分量。

（三）策略生命周期

参赛者必须暴露的唯一工厂函数与策略方法为：

```
def build_policy(context: BuildContext) -> Policy: ...

class Policy(Protocol):
    def reset(self, context: EpisodeContext) -> None: ...
    def act(self, observation: AgentObservation) -> np.ndarray: ...
    def close(self) -> None: ...
```

正式生命周期为：校验提交 → 创建各机器人策略实例 → 加载模型 → 每局重置局部状态 → 多次调用动作函数 → 回合结束 → 清理实例。每局必须清除历史

观测、递归网络状态和动作缓存；不同机器人之间不得共享可变记忆，不得跨评测回合适应测试集。同一提交可以为不同编号加载不同策略，也可以共享相同模型参数，参数在评测过程中冻结。

（四）训练接口

训练环境沿用 PettingZoo 多智能体环境的并行约定 [2]，底层接口与 Gymnasium 规范保持一致 [3]：

```
env = make_training_env(config)
observations, infos = env.reset(seed=train_seed)
state = env.state()
next_obs, rewards, terminated, truncated, infos = env.step(actions)
```

训练环境的每个机器人观测和动作与正式协议完全一致。训练全局状态按固定顺序展平（总长度 $6A + 6B + 1$ ， $A = B = 8$ 时为 97 维），包含当前所有实体状态，仅供集中训练使用，正式策略进程接触不到它。训练奖励字典给每个机器人同一个团队奖励，累计回报每步只加一次团队奖励。参赛者可自行改变训练奖励、增加课程或使用规则预训练，所用方法应记入实验报告。

四、参赛要求

（一）参赛流程

本次比赛采用“统一模板、本地迭代、过程报分、集中提交、统一核验、成果归档”的组织方式，流程如下。

（1）**编号分配与开发分支**。组织方发布比赛仓库并为每位参赛者分配唯一编号（Pxxx）。参赛者在仓库的 `participant/Pxxx` 目录下创建本人编号目录，并建立以本人编号命名的开发分支开展开发；代码推送到比赛仓库，组织方可以看到全部开发过程。参赛期间仅可修改本人编号目录，不得修改任务环境、接口定义和评测程序。

（2）**本地开发与实验记录**。参赛者在本人分支上完成策略开发、模型训练与优化，通过 Git 保留提交历史。每次重要迭代记录修改内容、实验配置、测试结果及结论，统一维护 `LOG.md`（迭代与思路）、`experiments.csv`（实验数据）与 `REPORT.md`（最终方案与复现说明）。完整的迭代记录是评分的核心依据之一，统一提交窗口开放前不得向其他渠道发布比赛实现。

(3) **过程测试与成绩上报。**比赛期间可使用公开评测程序本地测试，并通过指定渠道单独上报成绩（参赛编号、方案版本、评测版本、成绩与时间）。组织方据此维护过程排行榜，标注“参赛者自行测试，未经组织方核验”；过程排行榜不作为最终排名依据。

(4) **最终提交。**组织方公布统一提交窗口。每位参赛者只能有一个有效的最终提交 PR；PR 标题必须为 [Pxxx] Final Submission，并指向本人编号分支的最终提交。截止时间前可以关闭旧 PR 并重新提交；截止后以最后一个满足规范且在截止时间前创建的 Final Submission PR 为准。

(5) **统一核验。**本赛事采用可信参赛环境下的轻量安全模型：提交内容在最终评分前经过自动静态检查和组织方人工审核；禁止使用网络访问、外部进程调用、动态代码执行以及不安全对象反序列化等行为；模型文件采用受控格式。核验时，组织方评分程序在统一的软件环境、依赖版本、随机种子和任务配置下重新执行各参赛者提交，以复核成绩作为最终成绩；参赛者自行上报的成绩不替代实际评测。无法正常运行、材料缺失或不符合接口要求的提交，按赛前公布的异常处理规则处置。

(6) **成绩公布与归档。**组织方完成核验与评审后，公布最终成绩、分项结果和排行榜，并受理成绩复核申请。参赛者的完整代码、模型及迭代历史保留在本仓库的本人编号目录与对应分支中，组织方统一维护比赛规则、排行榜和成绩档案，形成可查询、可追溯的参赛记录。

(二) 环境与运行要求

1. **双环境依赖：**评测环境依赖以锁定文件为准（`requirements-official.lock`，仅含仿真运行库，无深度学习框架）；训练环境依赖以训练锁定文件为准（`requirements-train.lock`，含 PyTorch、Stable-Baselines3 等 [4]）。训练完成后将网络权重导出为 `npz` 等纯数值数据格式，由 `entry.py` 在评测环境中以纯数值计算前向推理。
2. **模型安全：**模型文件采用受控数据格式（如 `npz`）；禁止提交依赖 Pickle 反序列化的模型文件，包括 `.pkl`、`.pickle`、`.pt`、`.pth` 等；提交源码中禁止 `pickle.load`、`torch.load` 等不安全反序列化调用；模型结构及推理逻辑必须以可审查的源代码形式提交。静态审计与人工审核发现违规即拒绝提交。

3. **推理约束：**推理统一在中央处理器上进行；协议 `ResourceLimits` 规定初始化、单步决策、单回合与整个任务四层时间限制；内存与模型体积限制独立配置。模型必须能在断网环境中加载。
4. **本地预检与预览：**提交前使用预检脚本完成自查；本地评分在非 Linux 平台自动降级为预览模式，正式成绩一律以组织方 Linux 容器评测为准。

五、提交材料

每个参赛编号目录至少包含表 3 所列文件（模板与示例见仓库中的参赛模板目录 `participant/_template`，复制后把目录名与清单中的 `participant_id` 一并改为组织方分配的编号）。

表 3: 提交材料清单

文件	内容
<code>entry.py</code>	策略工厂入口（加载策略的唯一入口）
<code>submission.yaml</code>	协议、规则版本、入口、模型清单、推理依赖和复现命令
<code>artifacts/</code>	正式评分实际使用的模型、归一化统计与必要配置
<code>requirements-infer.lock</code>	推理依赖的确定版本，不得替换任务环境与接口包
<code>train.py</code> 或等价入口	学习模型的训练命令
<code>LOG.md</code>	简要实验过程和决策记录
<code>experiments.csv</code>	可比较的实验数据
<code>REPORT.md</code>	最终方案、对照结果、局限与复现说明
<code>THIRD_PARTY.md</code>	第三方代码、模型和许可证信息
<code>LICENSE</code>	本人可授权部分的许可证

提交清单（`submission.yaml`）的关键字段包括：参赛编号、入口（`entry: build_policy`）、模型清单（路径与 SHA-256 内容摘要）、推理依赖锁定文件与训练命令；协议版本为 `coverage-policy/1.0`，任务版本为 `coverage-task/1.0`。清单使用编号目录内的相对路径；跳出目录的路径和外部可变模型下载会被拒绝。

原创代码采用 MIT 许可证作为事先明确的参赛公开约定；提交时明确确认本人可授权代码的公开许可，第三方代码保留原许可证与署名。欢迎发现并报告评测漏洞：有效漏洞及其复现过程作为额外能力评价依据；任务性能成绩按漏洞修复后的复测结果认定。

六、评分标准

（一）指标定义

设第 t 步机器人—目标覆盖匹配数为 m_t ，目标总数为 M ，定义**覆盖率**

$$C_t = \frac{m_t}{M}. \quad (1)$$

设第 t 步参与至少一次机器人碰撞的机器人数量为 n_t ，定义**碰撞参与率**

$$K_t = \frac{n_t}{N}. \quad (2)$$

该统计不因同一对机器人被双向遍历而重复计数；去重后的碰撞对数另行保存用于诊断。

（二）团队奖励与回合回报

团队奖励定义为覆盖率与碰撞参与率的加权差：

$$r_t = C_t - \lambda K_t, \quad \lambda = 0.2, \quad (3)$$

其中碰撞权重 λ 为冻结配置。回合累计回报与单位时间回报分别为

$$R = \sum_{t=1}^T r_t, \quad J = \frac{R}{T}. \quad (4)$$

每步回报取值范围为 $[-\lambda, 1]$ 。固定长度回合下优化 R 与 J 等价；跨不同回合长度聚合时使用 J ，避免通过更长回合获得更高分。训练中允许加入“接近未覆盖目标”等辅助奖励项帮助学习，但辅助项不进入计分回报 J 。

（三）分层评分公式

最终成绩采用**三层结构**：第一层为逐回合的覆盖—碰撞奖励（式 (3)），第二层为回合与组内聚合（式 (4)），第三层为跨难度分组的基准相对归一化。评分版

本 coverage-score/1.0 对基础组与协作组分别冻结随机策略锚点 b_k 与学习策略锚点 a_k ，最终性能分定义为

$$S = 1000 \times \sum_k w_k \cdot \frac{\bar{J}_k - b_k}{a_k - b_k}, \quad \sum_k w_k = 1, \quad (5)$$

其中 $\bar{J}_k$ 为第 k 组全部回合单位时间回报的均值， w_k 为组权重。

本届公开套件 v1 的冻结参数为：基础组与协作组各权重 $w_1 = w_2 = 0.5$ ，锚点 $b_k = 0$ 、 $a_k = 1$ ，回合长度 $T = 10$ 。此时式 (5) 数值上等价于“场均回报 $\times 100$ ”。锚点从赛前独立校准产生并公布，不使用参赛者成绩动态更新，也不从私有最终测试重新计算。所有参赛者使用相同的场景种子与策略重复采样安排；排名过近时按赛前公布的统一补测规则处理。

（四）难度分组

公开评测套件 v1 包含两个难度分组、各 2 个场景、每组 2 次策略重复，共 8 个回合：

分组	配置	考察能力
基础组	$N = M = 3$ ，较宽视野、低速目标	跑通学习与基本分工
协作组	$N = M = 3$ ，更具挑战的运动与视野参数	局部协作、跟踪、避碰

正式测试隐藏具体种子与生成的场景，不临时引入未声明的新规则；公开测试与私有测试种子不重合，精英方案选择与调参只使用训练及公开测试数据。

（五）基线与成绩锚点

赛事发布三档基线作为评分刻度：随机策略基线（验证评测流程可用）、合法局部规则策略基线（检验任务区分度）与学习基准（1M 步 PPO 训练 [5][4]，证明公布资源预算内可完成有效学习循环）。三基线于 2026 年 9 月 19 日完成正式运行，成绩见表 4。

随机与规则基线拉开约 3 倍差距，任务区分度下限成立。规则基线显著高于学习基准属如实记录：学习基准仅在单一场景上训练，且训练奖励与计分标准之间存在适配间隙；参赛者完成跨场景训练并使训练奖励与计分标准一致之后，成绩即有明显超越空间。

表 4: 基线正式运行成绩（公开套件 v1, 8 回合）

基线	定位	性能分 S
P901	随机策略	58.33
P902	局部规则策略	175.00
P903	学习基准（1M 步 PPO）	116.67

（六）成绩类别与最终评定

比赛成绩按五类分别记录：**公开测试成绩**（参赛者自行上报）、**核验成绩**（组织方统一环境重评）、**实验质量成绩**、**工程质量成绩**与**最终成绩**。评分的两个核心依据是**完整的迭代记录**与**优良的测试分数**：前者来自 LOG.md、experiments.csv 与 REPORT.md 中的实验过程记录，后者来自统一复核后的性能分。先设置可复现、接口合格、交付完整的资格门槛，再发布三列：核验性能分（按式 (5) 计算）、实验质量等级与工程质量等级。算法榜按核验性能分排序。

实验与工程评价各设 0 至 5 个明确检查项，不按提交次数、代码行数或日志长度加分：实验检查对照设计、变量控制、重复实验、结论证据与失败分析；工程检查可运行入口、依赖锁定、模块组织、错误处理与复现说明。若采用加权总分，权重与判定标准赛前公布，结果保留各分项。

（七）异常处理

协议错误、推理超时与加载失败必须输出明确状态。正式有效分要求完整跑完规定测试，不允许删去失败回合后对剩余成绩求平均。参赛者实现问题标记为提交无效，保留诊断结果但不计正式分；基础设施故障按预先定义的重试规则重新运行。开赛后若修复影响行为的错误，组织方统一公告并提供固定新版本，安排所有受影响提交重测。

七、附则

（一）版本冻结

已冻结的数值以仓库配置目录为准：观测容量 $A = B = 8$ 、场地半宽、时间步长、回合时长、半径、驱动力、阻尼、最大速度、边界行为、目标运动分布、感知

范围、碰撞权重 $\lambda = 0.2$ 、难度权重、分数锚点、评测局数、策略重复次数、推理资源限制、复核误差容差与近分补测阈值。提交时间与时区由组织方赛前公布。

（二）扩展规则

更换强化学习算法或优化器、更换网络结构、增加递归记忆、使用集中价值网络、参数共享或角色策略、规则与学习组合、改变训练奖励与课程、在声明容量内扩展规模等改动均**不需要**修改既有接口；增加摄像头图像、通信动作或新动作维度属于新规则主版本，需重新定义与验证。协议版本、任务版本、评分版本与测试集版本分别管理，不同评分版本的分数不得直接混排。

（三）成果归档

统一提交后，参赛者的完整代码、模型及迭代历史保留在本仓库的本人编号目录与对应分支中；组织方统一维护比赛规则、排行榜和成绩档案，形成可查询、可追溯的参赛记录。

参考文献

参考文献

- [1] LOWE R, WU I, TAMAR A, et al. Multi-agent actor-critic for mixed cooperative-competitive environments[C]//Proceedings of the 31st International Conference on Neural Information Processing Systems. Long Beach: Curran Associates, 2017: 6379-6390.
- [2] TERRY J K, BLACK B, GNANESWAR N, et al. PettingZoo: gym for multi-agent reinforcement learning[C]//Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1. 2021: 1-14.
- [3] TOWERS M, TERRY J K, KWIATKOWSKI A, et al. Gymnasium: a standard interface for reinforcement learning environments[EB/OL]. (2024-07-24)[2026-09-20]. <https://arxiv.org/abs/2407.17032>.
- [4] RAFFIN A, HILL A, GLEAVE A, et al. Stable-baselines3: reliable reinforcement learning implementations[J]. Journal of Machine Learning Research, 2021, 22(268): 1-8.
- [5] SCHULMAN J, WOLSKI F, DHARIWAL P, et al. Proximal policy optimization algorithms[EB/OL]. (2017-08-28)[2026-09-20]. <https://arxiv.org/abs/1707.06347>.